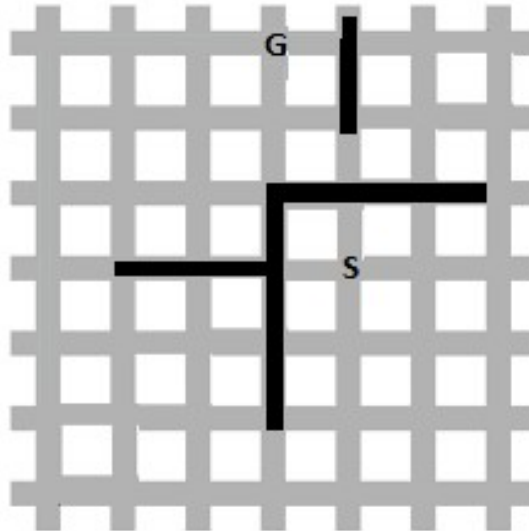


National University of Computer and Emerging Sciences, Lahore Campus

	Course Name:	Artificial Intelligence	Course Code:	AI 2002
	Program:	BS (CS)	Semester:	Spring 2025
	Duration:		Total Marks:	
	Paper Date:	Due Before final exam day	Weight	4%
	Section:	BCS 6A, 6B	Page(s):	
	Exam Type:	Final Assignment		

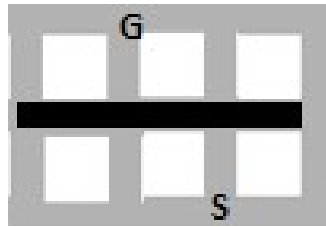
Q No 1 [Search Algorithms]

A company needs to write a program for an autonomous vehicle to plan a path from a starting point **S** to a destination **G** in a block town (a sample is shown below).

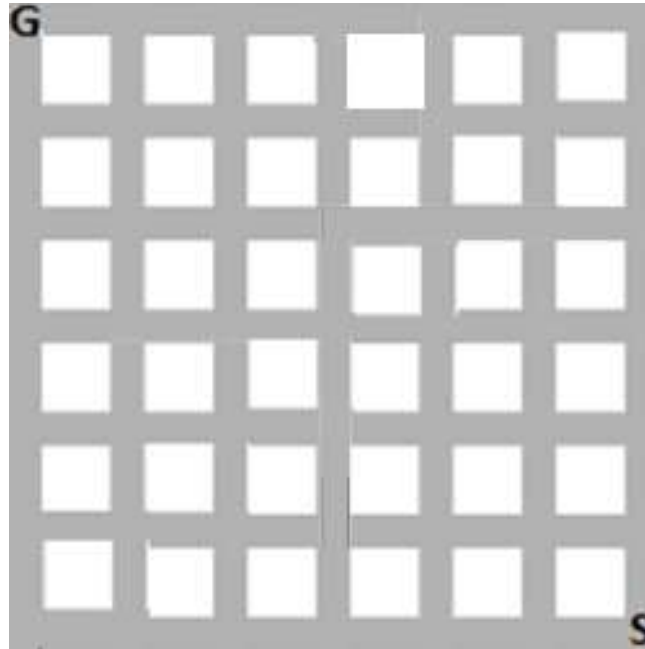


Each crossing in the town is connected to at most 4 other crossings. The CEO of the company (most probably a FAST-LHR alumni) decided to use A* algorithm for planning the path. Some roads in the town are closed for maintenance and are marked using **BLACK** color. Further, the distance between neighboring crossings is not same so some neighboring crossings are close whereas some are far from each other (The above picture does not capture this situation). For simplicity you can assume that the state of the entire road network is completely visible at all times and that we only need to plan paths between various crossings of the town. Answer the following questions for this problem.

- How would you represent state of the vehicle during the path planning?
- Describe at least two different heuristic functions that can be used to guide the A* search algorithms.
- For a simple town shown below show complete working of A* algorithm to find a path from S to G. You might assume that all crossings are equidistant from the neighboring crossings. You must show the state of Queue after the expansion of each node that comes out of queue during the algorithm execution.

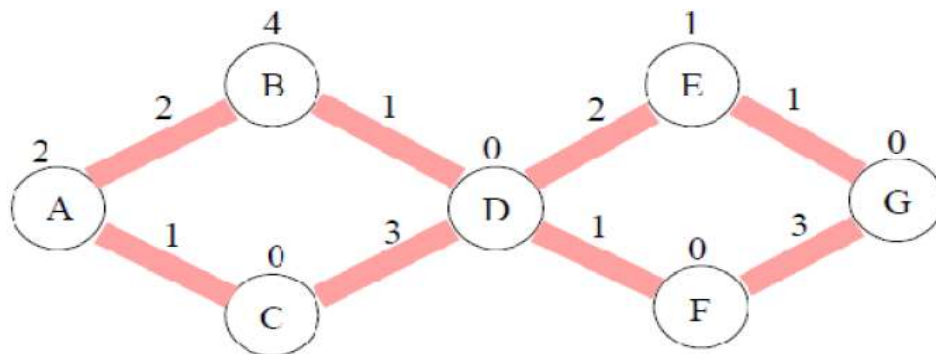


- d) For the following starting state compute an estimate of minimum number of nodes expanded by A*, BFS, DFS.



Q No 2 [Search Algorithms]

The following is a state-space search graph of a problem with nodes labeled using alphabets, edges are the thick shaded lines. The number above each edge giving the transition cost (e.g., $\text{cost}(C,D) = 3$) of the corresponding action and the number above each node is its heuristic value of that node (e.g., $h(A) = 2$). The node labeled A is the initial state and that labeled as G is the goal state.



- a) List, in order, the nodes that will be expanded if A* algorithm is used to find a path from the initial state to the goal state.
- b) What path (if any) would be found by A* algorithm in part a.
- c) What would be the state (nodes in it) of frontier/queue when the goal is found by A* algorithm in part b.
- d) In one of my implementation of A* I made an interesting mistake. My implementation was identical to the correct A*, except that when it visits a node n that has already been expanded, it immediately skips n instead of checking if it needs to reinsert n into the priority queue. What path would be found by my erroneous implementation of A* in the search problem given in part a above. Also comment on the effects of this change on the optimality of A* algorithm.
- e) One FAST- student suggested an early termination strategy for A* algorithm. He suggested that A* must be stopped as soon as the first goal node G is found in some successor list instead of waiting until G is popped off the priority queue. What path would be found by his version of A* for the state space search problem given in part a. Also comment on the effects of this change on completeness and optimality of A* algorithm

Q No 3 [Local Search]:

An intelligent but greedy automated agent/program is trying to solve the famous 0-1 knapsack problem. In this problem the agent has a set of items to choose from with each item having a weight the agent has to carry if it picks that item, and a value the agent will gain by picking up the item. The agent has the capacity to carry only a **MAX_WEIGHT** while his job is to select a subset of items that it can carry such that the total value gained by the agent is by picking up the subset of items is maximum.

The agent has been programmed by a selected team of students from a famous University (FAST) situated in the city of Lahore. The team programmed the agent such that it uses hill-climbing strategy to solve such optimization problems.

To represent state of the agent, while it is creating a plan to make a selection from a set of n items, the team used an array of size n with an index corresponding to one of the item in the set. Each index in the state array contains either a 0 or a 1 where 0 at any index means that the agent will not pick the corresponding item and a 1 means the agent will pick the corresponding item.

Further, to generate successors of a state a very simple successor function is used that works by selecting an index and if the corresponding item is not already in the list of items to be picked it added it into the list by placing a 1 at the corresponding array index.

- a) If the state of items contains 100 items
- What is the size of search space the agent has to search from? (Justify)
 - How many successors a state can have at max? (Justify)
 - What will be a suitable evaluation function to carry out the hill-climbing search?
- b) Now suppose that the set of items contain only 4 items. The weights of items and their value is given in the following table.

Item ID	A	B	C	D
Item Weight	1 kg	1 kg	2 kg	3 kg
Item Value	Rs. 100	Rs. 100	Rs. 300	Rs. 300

Table 1: Item Weights and Value

If the maximum capacity of the agent is 4 kg. and it starts in the following state then what will be final solution found be the agent (Show Complete Working)

Initial State	0	0	0	0
---------------	---	---	---	---

Q No 4 [Perceptron and Perceptron Learning]

Figure 1 show a single perceptron with two inputs x_1 and x_2 and a bias term that is permanently set to.

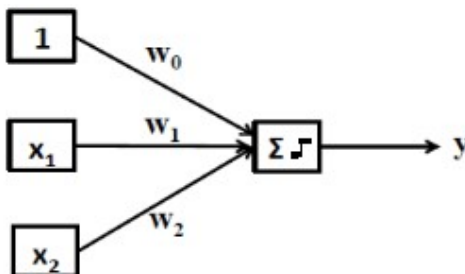


Figure 1

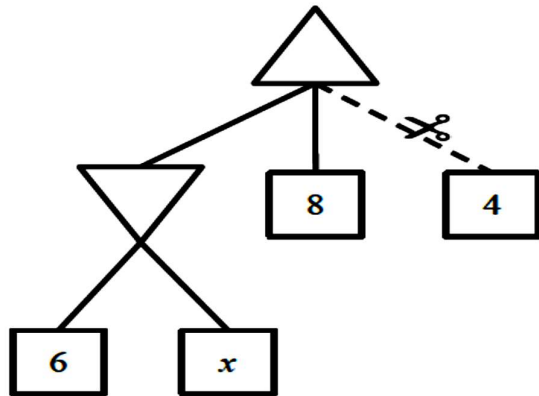
Output of this perceptron is computed using a simple threshold and is given as

$$y = \begin{cases} 1 & \text{if } (w_0 + w_1 \cdot x_1 + w_2 \cdot x_2) \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

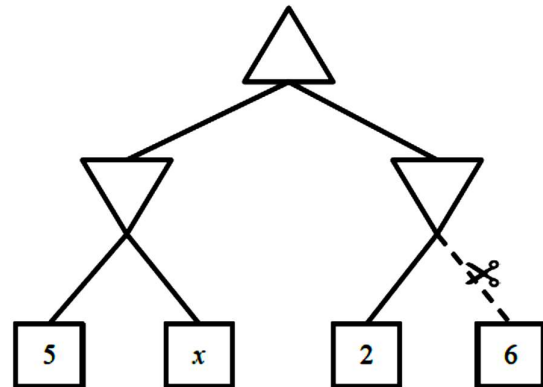
- It is desirable to learn weights $[w_0 \ w_1 \ w_2]$ of a perceptron that is similar to the perceptron in Figure 1. This perceptron will have two real valued inputs x_1 and x_2 and a bias term set permanently to 1. The objective is to learn weights such that perceptron gives a 1 as output if $x_2 \geq x_1$ and -1 otherwise. In this part you are required to create training data that can be used to learn weights of the perceptron. The training data must consist of two positive examples (i.e. examples for which output must be 1) and two negative examples (i.e. examples for which output must be -1)
- Use perceptron learning rule to learn weights $[w_0 \ w_1 \ w_2]$ of the perceptron using your training data of part a. You must start with initial weights $[0 \ 0 \ 0]$ and only do two iteration/epochs for

Q No 5 [Game Playing]

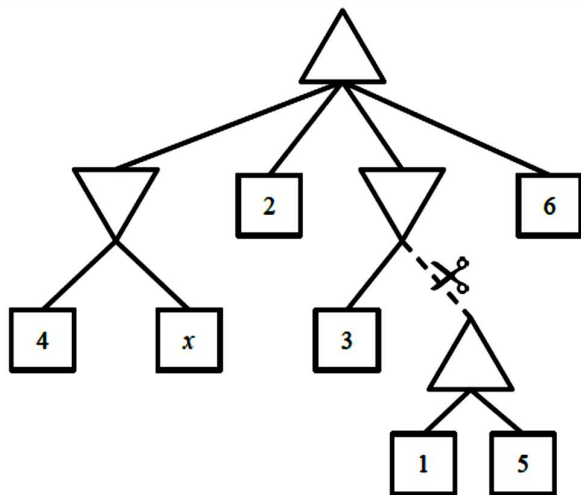
For each of the game-trees shown below, state for which values of x the dashed branch with the scissors will be pruned. If the pruning will not happen for any value of x write \none". If pruning will happen for all values of x write \all".



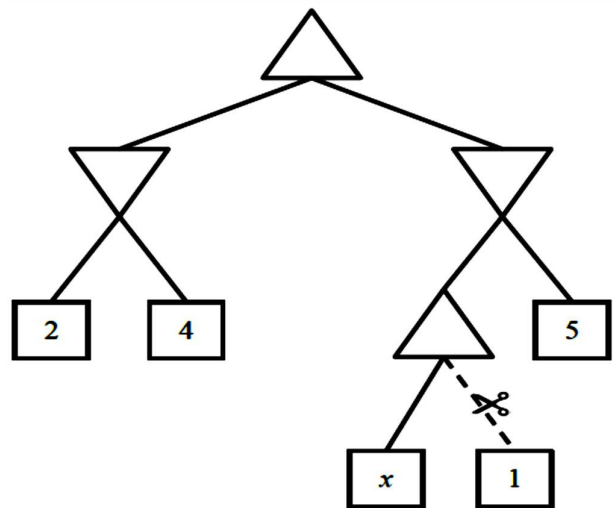
X _____



X _____



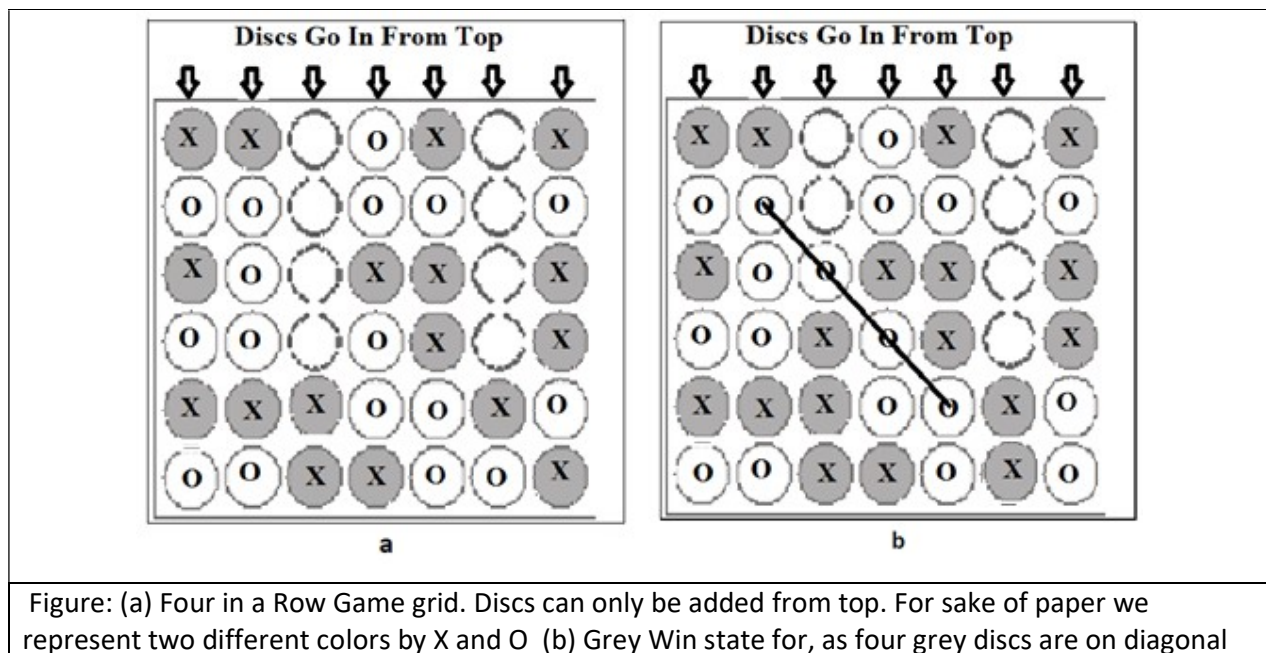
X _____



X _____

Q No 6 [Game Playing: Four In A Row]

Four in a Row is a two-player connection game in which the players first choose a color and then take turns dropping colored discs from the top into a seven-column, six-row vertically suspended grid. (As shown in figure 1a) The objective of the game is to be the first to form a horizontal, vertical, or diagonal line of four of one's own discs before your opponent.



The rules for Four in a Row are simple.

- The field (board) has seven columns and six rows.
- Two players play by alternately dropping a chip down one of the columns (from top).
- The chip drops to the lowest unoccupied spot in that column.
- The first player to get four of his own chips in a row, either vertical, horizontal, or diagonal, wins.
- The game ends in a draw if it fills before someone wins.

An AI student has decided to build an automatic player of FOUR IN A ROW using MINIMAX algorithm. Initially he decide to calculate a move at any given point in the game by building a complete game tree.

a) How many nodes will the game tree have when making the first move?

(Give an approximate Answer) Note that at each level a player has about five possible moves on average

The student figured out that the number of nodes in the game tree is large enough to prohibit building a complete game tree therefore he decided to choose a move by looking only D level deep in the tree. For this purpose he comes up with the following evaluation/expert function E.

```

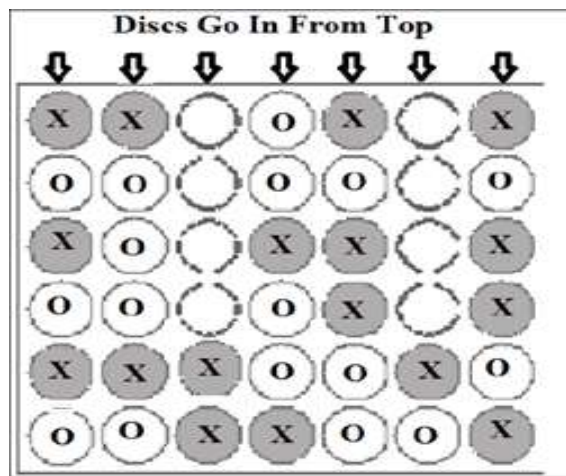
int[][] evaluationTable = {{3, 4, 5, 7, 5, 4, 3},
                           {4, 6, 8, 10, 8, 6, 4},
                           {5, 8, 11, 13, 11, 8, 5},
                           {5, 8, 11, 13, 11, 8, 5},
                           {4, 6, 8, 10, 8, 6, 4},
                           {3, 4, 5, 7, 5, 4, 3}};
//This evaluation table is used as follows

int evaluateContent() {
    int utility = 128;
    int sum = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < columns; j++)
            if (board[i][j] == 'O')
                sum -= evaluationTable[i][j];
            else if (board[i][j] == 'X')
                sum += evaluationTable[i][j];
    return utility + sum;
}

```

The main idea behind this evaluation function is that the numbers in the table indicate the number of four connected positions which include that space. This gives a measurement of how useful each square is for winning the game and hence it helps decide the strategy. The student implemented the MINIMAX (with alpha-beta pruning) algorithm using his evaluation function. For the state of game given in figure 2.

- b) For the game state shown below where it is turn of X to make a move show which move will be selected by the MINIMAX if $D = 2$ i.e. depth of at most 3 is used to select a move. Show complete game tree used by the player and also show all pruned nodes.



Q No 7 [Search Solution Using GA]

Subset-sum is an important NP-complete decision problem in computer science and can be stated as follows

For a multi-set of integers, is there a non-empty subset whose sum is exactly equal to a given value S.

For example, given the multi-set $\{-7, -7, -3, -2, 9000, 5, 8\}$ of seven integers and the value -19, the answer to the subset sum problem is **YES** because the multi-subset $\{-7, -7, -3, -2\}$ sums to -19.

Similarly for the given set $\{-7, -3, -2, 9000, 5, 8\}$ of six integers and the value -5, the answer to the subset sum problem is **YES** because several subset exists that sums to -5 like $\{-3, -2\}$ and $\{-7, -3, 5\}$ etc.

Furthermore for the same set $\{-7, -3, -2, 9000, 5, 8\}$ of six integers and the value 18000, the answer to the subset sum problem is **NO** because no subset exists that sums to 18000.

For a set having n-elements we can represent the subset sum problem as an evolutionary search problem using the following encoding of solution/chromosome

Chromosome: is a bit vector of length n with a bit being set to 1 if the element is part of the subset and 0 otherwise.

For example, for the multi-set $\{-7, -7, -3, -2, 9000, 5, 8\}$ the chromosome will be a bit vector of length 7, hence the chromosome 1111000 will represent the solution/subset $\{-7, -7, -3, -2\}$. Similarly the chromosome 1111111 represents the solution/subset $\{-7, -7, -3, -2, 9000, 5, 8\}$

Chromosome Fitness: The fitness of a solution can then be computed using $1 / (|S - \sum| + 1)$ where S is the required value of sum and $\sum$ is sum of the subset and $|x|$ represents absolute value of x.

For example, for the multi-set $\{-7, -7, -3, -2, 9000, 5, 8\}$ and $S = -19$ the chromosome 1111000 has fitness value of 1 whereas the chromosome 1111111 has fitness 1/8995. Verify that you understand the formula for computing fitness.

Assume that in an implementation of GA is being used to find a subset of the set $\{2, 3, 4, 8, 16\}$ having sum 17 and that the following five solutions have been randomly generated as the initial population.

$\{11001, 10001, 00111, 011111, 00010\}$

Part a) Compute fitness of each solution in this population using the formula given above.

Part b) Use fitness proportionate selection to select 1 elite/solution that must be transferred unchanged to the next generation of solutions.

Part c) Generate the remaining chromosomes/solutions of the next generation using single point crossover and mutation operation. Here assume that GA uses fitness proportionate selection and that the mutation rate is 0.001. Points will be awarded for correct selection (fitness-proportionate), correct crossover and correct mutation and correct computation of next generation

Whenever you need a random number select the next random number from the following **list/2D-array** of randomly generated numbers and if the list exhausts and you still need more random numbers then you must start choosing the number from the beginning (**row 0, col 0**) of the list again.

To make the things a bit more interesting and more specific to each students, you are required to **start choosing the random numbers from different place in the list** based on the last digit of your registration number.

So if the last digit of your registration number (XXX-XXX0) is 0 you must start at position (0, 0), shown in bold and green color, and move on row-wise until all rows have been used.

Similarly if the last digit of your registration number (XXX-XXX5) is 5 then you must begin at position (0, 5), shown in bold and red color, and move on till the end.

R\C	0	1	2	3	4	5	6	7	8	9
0	0.487	0.937	0.291	0.816	0.051	0.621	0.248	0.691	0.882	0.025
1	0.038	0.096	0.322	0.179	0.967	0.064	0.930	0.014	0.091	0.844
2	0.766	0.358	0.797	0.609	0.846	0.498	0.612	0.476	0.946	0.873
3	0.757	0.944	0.749	0.966	0.033	0.755	0.659	0.161	0.684	0.095

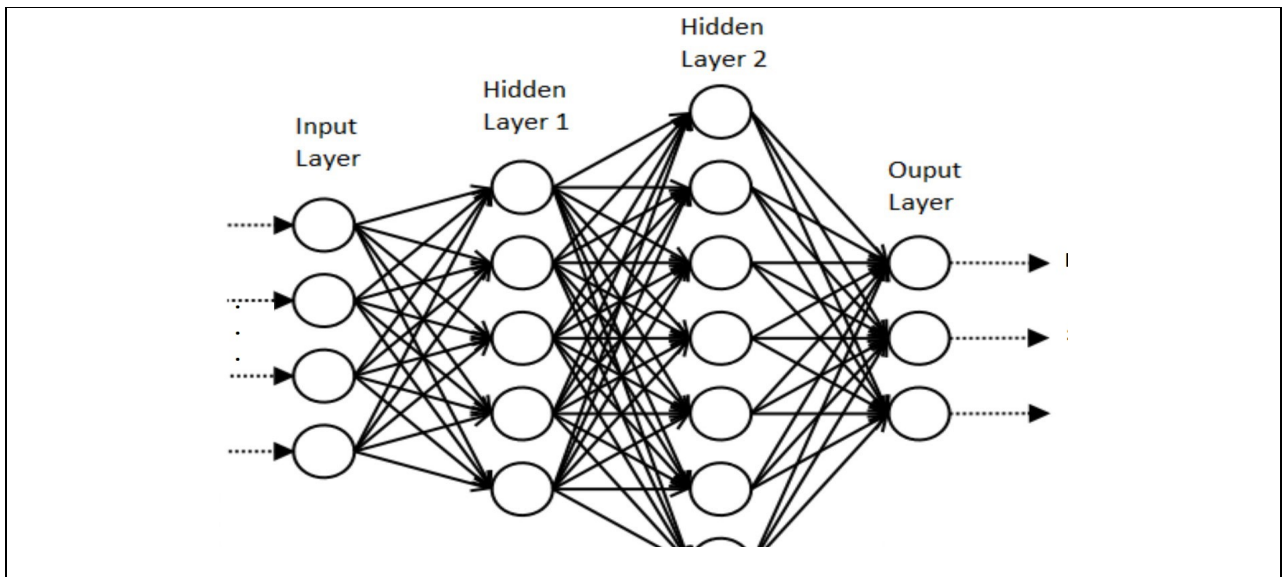
Q No 9 [Neural Networks]

Inspired by the movie WALL-E a teacher at FAST-LHR is designing a line following robot. You might search/google Internet to see various line following robots but don't spend much time surfing the Internet as this is a limited time Exam.

Being aware of the representation and learning power of a feed-forward artificial neural network, he decided to use such a neural network with two hidden layers of neurons for creating the main controller/brain of the robot. The initial prototype of the robot will only follow smooth lines i.e. lines without any sharp turns.

Input to the controller/brain of this robot will be a grey-scale camera image of dimensions **N x N**. The number of neurons in the first hidden layer will be **H1** and the number of neurons in the second hidden layer will be **H2** whereas there will be **3** neurons in the output layer with each output being **between 0 and 1** specifying the direction in which the robot need to turn. The robot will be turned slightly towards left, right or will go straight depending upon the largest value of the output neuron **Left, Straight or Right** respectively.

Such a neural network is depicted in the figure below



In this question we will assume that **N** is equal to the last four digit number in you university ID whereas **H1** is the number consisting of the year/batch number in your ID and **H2** is the most significant two digits of **N**. For example if your id is L17-4526 then N is 4526, H1 is 17 and H2 is 45.

Part a) [Neural Network Weights]

Specify the number of weights needed to create such a neural network. Assume that each neuron in this network has a bias term as well. Further if we assume that each weight is a double precision number represented using 8-bytes, approximately how many kilo bytes will be needed to store this neural network on some storage.

Part b) [Neural Network Computations]

Assuming that the camera is generating 30 frames/Images per second and the neural network will be used to make a decision for each one of the frames, how many floating point multiplications and additions will be computed per second if this network is used for making decision.

Part c) [Preparing Training Data]

We know that weights of a neural network are typically learned using back-propagation learning algorithm which is a supervised learning algorithm.

- i) What do we mean by supervised learning? Give names of three supervised learning algorithms for classification.
- ii) Briefly describe some method to prepare training data for learning weights of the network described in the previous parts.

Part d) [Neural Network Training]

- iii) Use backpropagation learning to learn weights of a neural network

Q No 10 [Deep Learning: CNN]

You come up with a CNN classifier. For each layer, calculate the number of weights, number of biases and the size of the associated feature maps.

The notation follows the convention:

- CONV- K - N denotes a convolutional layer with N filters, each them of size $K \times K$, Padding and stride parameters are always 0 and 1 respectively.
- POOL- K indicates a $K \times K$ pooling layer with stride K and padding 0.
- FC- N stands for a fully-connected layer with N neurons.

Layer	Activation map dimensions	Number of weights	Number of biases
INPUT	$128 \times 128 \times 3$	0	0
CONV-9-32			
POOL-2			
CONV-5-64			
POOL-2			
CONV-5-64			
POOL-2			
FC-3			

Following topics are also included in the final exam

Clustering (K-means and Agglomerative)